MathViz 3D — API Gateway & Data Layer Build Guide

> **Scope:** This document covers the API Gateway (authentication, rate limiting, routing, WebSocket upgrade) and the Data Layer (PostgreSQL schema, Redis key patterns, migrations, and backup strategy). These are the final two infrastructure components required to complete the MathViz 3D system.

Part 1 — API Gateway

1. Purpose and Role

The API Gateway is the single entry point for all external traffic — browser clients, mobile clients, and any third-party API consumers. It sits in front of the LLM Service and Visualization Service and is responsible for:

- TLS termination
- User authentication (JWT validation)
- Rate limiting (per user, per tier)
- Request routing to the correct internal service
- WebSocket upgrade and proxy
- CORS enforcement
- Request/response logging for audit and analytics

The Gateway contains no business logic. It does not know what a Scene Description is. It routes, validates identity, enforces limits, and passes through.

---

2. Technology Choice

The Gateway is implemented with **Traefik v3** for production deployments and **Nginx** as an alternative for simpler environments. Both are configured via declarative files rather than custom code.

For the WebSocket authentication middleware and JWT validation, a small **FastAPI middleware service** (the "Auth Sidecar") handles logic that cannot be expressed in Traefik/Nginx config alone. The Auth Sidecar is stateless and horizontally scalable.

```
gateway/
├── traefik/
│   ├── traefik.yml            # Static Traefik configuration
│   ├── dynamic/
│   │   ├── routers.yml        # Route definitions
│   │   ├── middlewares.yml    # Rate limiting, headers, CORS
│   │   └── services.yml       # Internal service targets
│   └── tls/
│       └── certificates.yml   # TLS cert resolver (Let's Encrypt)
│
├── auth_sidecar/
│   ├── main.py                # FastAPI app: /auth/verify, /auth/ws-token
│   ├── jwt.py                 # JWT validation and claims extraction
│   ├── tokens.py              # Short-lived WebSocket token issuance
│   ├── config.py              # Settings: JWT secret, token TTL, Redis URL
│   └── tests/
│       ├── test_jwt.py
│       └── test_tokens.py
│
└── nginx/                     # Alternative config for simpler deployments
    └── nginx.conf
```

---

3. Routing

3.1 Route Table

Incoming Path	Method	Target Service	Notes
`/api/chat`	POST	LLM Service `:8001/chat`	Auth required
`/api/sessions/{id}/history`	GET	LLM Service `:8001/sessions/{id}/history`	Auth required
`/api/sessions/{id}`	DELETE	LLM Service `:8001/sessions/{id}`	Auth required
`/api/scenes`	POST	Visualization Service `:8002/scenes`	Internal only (blocked externally)
`/api/scenes/{id}/steps/{n}`	GET	Visualization Service `:8002/scenes/{id}/steps/{n}`	Auth required

Incoming Path	Method	Target Service	Notes
`/api/scenes/{id}/state`	GET	Visualization Service `:8002/scenes/{id}/state`	Auth required
`/api/scenes/{id}/reset`	POST	Visualization Service `:8002/scenes/{id}/reset`	Auth required
`/api/scenes/{id}/stream`	WS	Visualization Service `:8002/scenes/{id}/stream`	WS upgrade; token auth
`/api/tasks/{id}`	GET	Visualization Service `:8002/tasks/{id}`	Auth required
`/auth/login`	POST	Auth Sidecar `:8003/auth/login`	Public
`/auth/refresh`	POST	Auth Sidecar `:8003/auth/refresh`	Refresh token required
`/auth/ws-token`	POST	Auth Sidecar `:8003/auth/ws-token`	Auth required
`/health`	GET	Gateway itself	Public; returns 200

`POST /api/scenes` is blocked at the gateway for external callers. Only the LLM Service (identified by `X-Internal-Secret` header) may POST to it. The gateway enforces this by checking for the internal secret header and rejecting requests without it from non-internal IP ranges.

3.2 Traefik Router Configuration

Each route is a Traefik router with:

- A rule matching the path prefix and method.
- A middleware chain applied in order: `cors → rate-limit → auth-forward → strip-prefix`.
- A service target pointing to the internal container hostname and port.

WebSocket routes additionally have: `sticky sessions` disabled (WebSocket connections are stateless after auth), `timeout` set to 24 hours (the max session lifetime), and the `Upgrade` header preserved through the proxy.

---

4. Authentication

4.1 JWT Structure

MathViz 3D uses short-lived JWTs (access tokens) and long-lived refresh tokens.

Access token claims:

```json
{
  "sub": "user_uuid",
  "session_id": "session_uuid",
  "plan": "free | pro | enterprise",
  "iat": 1700000000,
  "exp": 1700003600
}
```

Access token TTL: **1 hour**. Signed with HS256 using a secret stored in the secrets manager. The secret rotates every 30 days — a configurable grace window (5 minutes) allows tokens signed with the previous key to remain valid during rotation.

Refresh token: an opaque random 256-bit token stored in Redis at key `refresh:{token_hash}` with a 30-day TTL. Invalidated on logout or password change.

4.2 Auth Sidecar (`auth_sidecar/`)

`POST /auth/login` Accepts `{email, password}`. Looks up the user in PostgreSQL. Verifies the bcrypt password hash. Issues a new access token and refresh token. Stores the refresh token hash in Redis. Returns both tokens.

`POST /auth/refresh` Accepts `{refresh_token}`. Hashes the token, looks it up in Redis. If valid, issues a new access token (and optionally a new refresh token — rolling refresh). Returns the new access token.

`POST /auth/ws-token` (Auth required via Bearer token) Issues a short-lived (60 s) signed WebSocket token containing: `user_id`, `session_id`, `scene_id`, `expires_at`. This token is passed as a query parameter when opening the WebSocket connection (`/api/scenes/{id}/stream?token=...`). The Visualization Service validates it independently. The short TTL limits the exposure window if the token is captured in a URL log.

`GET /auth/verify` (Internal — called by Traefik ForwardAuth middleware) Traefik's `forwardAuth` middleware calls this endpoint for every protected route before proxying the request. It validates the `Authorization: Bearer {token}` header and returns:

- `200 OK` with `X-User-Id` and `X-Session-Id` headers (forwarded to the target service) if valid.
- `401 Unauthorized` if the token is missing, expired, or invalid. Traefik returns 401 to the client without proxying.

The `X-User-Id` and `X-Session-Id` headers injected by the gateway are trusted by the LLM Service and Visualization Service — they never re-validate the JWT themselves, relying entirely on the gateway's ForwardAuth.

4.3 JWT Validation (`jwt.py`)

Use `python-jose` or `PyJWT` for JWT decode and verification.

Validation steps:

1. Decode the token with the current signing secret. If it fails, try the previous secret (rotation grace window).
2. Check `exp` — reject if expired (with no grace).
3. Check `iat` — reject if issued in the future (clock skew tolerance: 30 s).
4. Check that `sub` (user_id) is a valid UUID format.
5. Check that `plan` is one of the known plan tiers.

Any failure raises `AuthenticationError`, which the sidecar maps to a 401 response.

---

5. Rate Limiting

Rate limits are applied per user, keyed by `user_id` from the JWT claims. Unauthenticated requests (to public endpoints) are keyed by IP address.

5.1 Limit Tiers

Tier	LLM Requests/min	LLM Requests/day	Scene Renders/day	WebSocket Connections
Free	5	50	20	1 concurrent
Pro	20	500	200	5 concurrent
Enterprise	100	Unlimited	Unlimited	50 concurrent

5.2 Implementation

Traefik's built-in `rateLimit` middleware handles per-IP limits for public endpoints. For per-user, per-tier limits (which require reading JWT claims), the Auth Sidecar's `/auth/verify` endpoint checks Redis counters and returns `429 Too Many Requests` when limits are exceeded. Traefik's ForwardAuth returns this status to the client without modification.

Redis counters:

- `rl:{user_id}:llm:min` — TTL 60 s, incremented on each LLM request.
- `rl:{user_id}:llm:day` — TTL 86400 s.

- `rl:{user_id}:renders:day` — TTL 86400 s.

Counter increment uses a Lua script for atomicity: `INCR key; EXPIRE key ttl; GET key` — all in one round trip. If the incremented value exceeds the tier limit, return 429 without proxying.

On 429 responses, include `Retry-After` and `X-RateLimit-Reset` headers populated from the Redis key TTL.

---

6. CORS Configuration

CORS is handled by a Traefik middleware applied to all `/api/*` routes.

Allowed origins: configured as an environment variable list (e.g., `https://mathviz.app`, `https://www.mathviz.app`, `http://localhost:5173` for development). No wildcard origins in production.

Allowed methods: `GET, POST, DELETE, OPTIONS`.

Allowed headers: `Authorization, Content-Type, X-Session-Id`.

Exposed headers: `X-RateLimit-Remaining, X-RateLimit-Reset, Retry-After`.

Preflight cache: `Access-Control-Max-Age: 86400` (cache preflight responses for 24 hours to reduce OPTIONS request volume).

WebSocket routes do not participate in CORS (WebSocket upgrades are not subject to the same-origin policy in the same way) — token-based auth is the security mechanism there.

---

7. TLS

Traefik manages TLS termination with Let's Encrypt via the ACME HTTP-01 or DNS-01 challenge. Certificates are stored in a JSON file on a persistent volume (or in a secrets manager in production).

TLS settings:

- Minimum TLS version: 1.2. Prefer TLS 1.3.
- Cipher suites: follow Mozilla's "Intermediate" profile.
- HSTS header: `Strict-Transport-Security: max-age=31536000; includeSubDomains`.

All HTTP traffic is redirected to HTTPS via a permanent redirect middleware applied at the entrypoint level.

---

8. Request Logging and Audit

All gateway access logs are written in JSON format to stdout (captured by the container runtime). Each log line includes: timestamp, method, path, response status, response time, `user_id` (if authenticated), `session_id`, request size, response size, and the upstream service target.

Sensitive fields (Authorization header values, request bodies) are never logged at the gateway. The `user_id` from JWT claims is logged for audit purposes but not the full token.

Logs are shipped to a centralized log aggregator (e.g., Loki, CloudWatch Logs) by the container runtime's log driver.

Part 2 — Data Layer

9. PostgreSQL Schema

The database stores persistent data: users, authentication sessions, conversations, scenes, step metadata, presets, and usage analytics. All tables use UUID primary keys. Timestamps are `TIMESTAMPTZ` (UTC).

9.1 Schema Overview

```sql
-- Users and authentication
CREATE TABLE users ( ... );
CREATE TABLE auth_sessions ( ... );

-- Conversations and messages
CREATE TABLE conversations ( ... );
CREATE TABLE messages ( ... );

-- Scenes and step state
CREATE TABLE scene_descriptions ( ... );
CREATE TABLE step_states ( ... );

-- Presets
CREATE TABLE presets ( ... );

-- Analytics
CREATE TABLE usage_events ( ... );
```

9.2 `users`

```sql
CREATE TABLE users (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email           TEXT NOT NULL UNIQUE,
    hashed_pw       TEXT NOT NULL,              -- bcrypt hash
    display_name    TEXT,
    plan            TEXT NOT NULL DEFAULT 'free'
                    CHECK (plan IN ('free', 'pro', 'enterprise')),
    prefs_json      JSONB NOT NULL DEFAULT '{}', -- UI preferences (theme, speed, etc.)
    email_verified  BOOLEAN NOT NULL DEFAULT FALSE,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    last_active_at  TIMESTAMPTZ,
    deleted_at      TIMESTAMPTZ                 -- soft delete
);

CREATE INDEX idx_users_email ON users (email) WHERE deleted_at IS NULL;
```

`prefs_json` stores user preferences that are not frequently queried: default camera preset, preferred step speed, HUD layout preferences. Queried only on session start — JSONB is appropriate here.

Soft delete: `deleted_at IS NOT NULL` marks a deleted account. All queries in the application layer include `WHERE deleted_at IS NULL` via SQLAlchemy's `with_expression` or a base query filter.

9.3 `auth_sessions`

Stores server-side session metadata. The actual refresh token is stored as a hash in Redis (for fast lookup and TTL management). This table is for audit and revocation.

```sql
```

```sql
CREATE TABLE auth_sessions (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id         UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    refresh_token_hash TEXT NOT NULL UNIQUE,    -- SHA-256 of the refresh token
    user_agent      TEXT,
    ip_address      INET,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    last_used_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    expires_at      TIMESTAMPTZ NOT NULL,
    revoked_at      TIMESTAMPTZ                 -- set on logout or password change
);

CREATE INDEX idx_auth_sessions_user ON auth_sessions (user_id);
CREATE INDEX idx_auth_sessions_token ON auth_sessions (refresh_token_hash)
    WHERE revoked_at IS NULL;
```

9.4 `conversations`

One row per LLM conversation session.

```sql
CREATE TABLE conversations (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id         UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    started_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    last_active_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    last_scene_id   UUID,                       -- FK to scene_descriptions, set after insert
    title           TEXT,                       -- auto-generated from first concept title
    ctx_summary     TEXT                        -- optional LLM-generated summary for pruned context
);

CREATE INDEX idx_conversations_user ON conversations (user_id, last_active_at DESC);
```

`last_scene_id` is a non-foreign-key reference (to avoid circular FK between conversations and scene_descriptions). It is updated by the application layer after a scene is inserted.

9.5 `messages`

One row per turn (user or assistant message).

```sql
```

```sql
CREATE TABLE messages (
    id                      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    conversation_id         UUID NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
    role                    TEXT NOT NULL CHECK (role IN ('user', 'assistant', 'system')),
    content                 TEXT NOT NULL,
    scene_description_json  JSONB,          -- populated for assistant turns that produced a scene
    scene_id                UUID,           -- FK to scene_descriptions (nullable)
    input_tokens            INTEGER,        -- LLM token usage for this turn
    output_tokens           INTEGER,
    provider                TEXT,           -- 'anthropic' | 'openai' | 'ollama'
    model                   TEXT,           -- e.g. 'claude-sonnet-4-20250514'
    created_at              TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_messages_conversation ON messages (conversation_id, created_at ASC);
CREATE INDEX idx_messages_scene ON messages (scene_id) WHERE scene_id IS NOT NULL;
```

`scene_description_json` stores the full Scene Description JSON that was forwarded to the Visualization Service. This is the source of truth for reconstructing a scene from history — the Redis cache may have expired.

9.6 `scene_descriptions`

One row per scene produced by the LLM.

```sql
```

```sql
CREATE TABLE scene_descriptions (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id         UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    conversation_id UUID REFERENCES conversations(id),
    scene_json      JSONB NOT NULL,             -- full SceneDescription JSON
    title           TEXT NOT NULL,              -- concept_title from SceneDescription
    tags            TEXT[] NOT NULL DEFAULT '{}',
    is_saved        BOOLEAN NOT NULL DEFAULT FALSE, -- user explicitly saved this scene
    is_public       BOOLEAN NOT NULL DEFAULT FALSE, -- shared publicly
    thumbnail_url   TEXT,                       -- generated after first render
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    last_viewed_at  TIMESTAMPTZ
);

CREATE INDEX idx_scenes_user ON scene_descriptions (user_id, created_at DESC);
CREATE INDEX idx_scenes_public ON scene_descriptions (is_public, created_at DESC)
    WHERE is_public = TRUE;
CREATE INDEX idx_scenes_tags ON scene_descriptions USING GIN (tags);
```

`tags` are auto-extracted from the `concept_type` values in the Scene Description (e.g., `['fourier', 'function_2d', 'series_convergence']`) plus any user-defined tags. The GIN index supports tag-based search.

9.7 `step_states`

Maps individual steps to their geometry cache keys. Used to reconstruct scenes from history when Redis has expired.

sql

```sql
CREATE TABLE step_states (
    id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    scene_id            UUID NOT NULL REFERENCES scene_descriptions(id) ON DELETE CASCADE,
    step_index          INTEGER NOT NULL,
    geometry_cache_key  TEXT,                   -- Redis key (may be NULL if not yet computed)
    concept_type        TEXT NOT NULL,
    expression          TEXT,                   -- primary expression for this step
    created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (scene_id, step_index)
);

CREATE INDEX idx_step_states_scene ON step_states (scene_id, step_index ASC);
```

`geometry_cache_key` is populated by the Visualization Service after computation. It can be NULL if the step has never been computed (new scene that hasn't been fully rendered yet).

9.8 `presets`

Built-in and community-contributed scene presets.

```sql
CREATE TABLE presets (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    category        TEXT NOT NULL,              -- e.g. 'calculus', 'linear_algebra', 'fourier'
    name            TEXT NOT NULL,
    description     TEXT,
    scene_json      JSONB NOT NULL,
    thumbnail_url   TEXT,
    author_id       UUID REFERENCES users(id),  -- NULL for built-in presets
    is_public       BOOLEAN NOT NULL DEFAULT TRUE,
    view_count      INTEGER NOT NULL DEFAULT 0,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_presets_category ON presets (category, view_count DESC);
CREATE INDEX idx_presets_public ON presets (is_public) WHERE is_public = TRUE;
```

Built-in presets are seeded via a migration. Community presets are submitted by pro/enterprise users and reviewed before `is_public` is set to true.

9.9 `usage_events`

Append-only analytics table. Never updated, never deleted (except by scheduled archival).

```sql
CREATE TABLE usage_events (
    id          BIGSERIAL PRIMARY KEY,          -- bigserial for high insert volume
    user_id     UUID NOT NULL,                  -- no FK — analytics should not cascade delete
    event_type  TEXT NOT NULL,                  -- 'llm_turn' | 'scene_render' | 'export' | etc.
    metadata    JSONB NOT NULL DEFAULT '{}',
    ts          TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_usage_user_ts ON usage_events (user_id, ts DESC);
CREATE INDEX idx_usage_type_ts ON usage_events (event_type, ts DESC);
```

No foreign key on `user_id` — analytics events must survive user deletion. `metadata` stores event-specific details: for `llm_turn`, it includes `{input_tokens, output_tokens, provider, model, scene_produced: bool}`. For `scene_render`, it includes `{scene_id, step_count, concept_types: [...]}`.

Partition this table by month using `PARTITION BY RANGE (ts)` once insert volume warrants it. Archive partitions older than 12 months to cold storage.

---

10. Redis Key Patterns

All Redis keys follow a consistent hierarchical pattern: `{namespace}:{identifier}:{qualifier}`. Keys never contain spaces, slashes, or quotes.

10.1 Full Key Reference

Key Pattern	Value Type	TTL	Owner	Purpose
`session:{session_id}`	Hash	24 h	Auth Sidecar	Auth session data: `user_id`, `plan`, `last_active`
`refresh:{token_hash}`	String	30 d	Auth Sidecar	Refresh token validity marker (value = `user_id`)
`rl:{user_id}:llm:min`	Counter	60 s	Auth Sidecar	LLM rate limit — per minute
`rl:{user_id}:llm:day`	Counter	86400 s	Auth Sidecar	LLM rate limit — per day
`rl:{user_id}:renders:day`	Counter	86400 s	Auth Sidecar	Render rate limit — per day
`llm_ctx:{session_id}`	List (JSON)	4 h	LLM Service	Conversation message history
`llm_session:{session_id}`	Hash (JSON)	4 h	LLM Service	ConversationState (concept history, token counts)
`geo:{hash}`	Binary (msgpack)	2 h	Viz Service	Geometry buffer cache keyed by (AST+domain) hash
`step_bundle:{scene_id}:{step}`	Binary (msgpack)	30 min	Viz Service	Pre-computed step bundle

Key Pattern	Value Type	TTL	Owner	Purpose
`scene:{scene_id}:state`	Hash (JSON)	30 min	Viz Service	Active scene playback state
`ws_token:{token_hash}`	String	60 s	Auth Sidecar	WebSocket short-lived token validity
`task:{task_id}:status`	Hash	24 h	Viz Service	Celery export task status for client polling

10.2 Key Design Principles

Namespacing: every key begins with a service-specific namespace (`rl:`, `geo:`, `llm_`, etc.). This allows Redis keyspace notifications to be filtered by namespace and prevents accidental key collisions between services.

TTL discipline: every key has an explicit TTL set at write time. No key is written without a TTL. Long-lived keys (refresh tokens, 30 d) are stored in a separate Redis logical database (`DB 1`) from short-lived operational keys (`DB 0`). This allows independent eviction policies: `DB 0` uses `allkeys-lru` (evict least-recently-used keys when memory is full, appropriate for cache); `DB 1` uses `noeviction` (never evict refresh tokens — they are security-critical).

Atomic operations: all counter increments use Lua scripts. All multi-key operations (e.g., checking a rate limit counter and incrementing it in one step) use transactions (`MULTI/EXEC`) or Lua to avoid race conditions.

Binary values: geometry buffers and step bundles are stored as raw msgpack bytes. Redis treats them as opaque binary strings. Never attempt to inspect or modify these values outside the Visualization Service — the msgpack schema is internal to that service.

---

11. Database Migrations

Migrations are managed with **Alembic** (SQLAlchemy's migration tool).

11.1 Migration Structure

```
alembic/
├── alembic.ini
├── env.py                  # Alembic env: reads DATABASE_URL, imports models
├── script.py.mako          # Migration file template
└── versions/
    ├── 0001_initial_schema.py
    ├── 0002_add_step_states.py
    ├── 0003_add_presets.py
    └── ...
```

11.2 Migration Conventions

- Every migration has both `upgrade()` and `downgrade()` functions. Downgrades must be tested.
- Migrations are never edited after they have been applied to production. New changes are always new migration files.
- Column additions are always `nullable` or have a `server_default` — never add a non-nullable column without a default to a table with existing data.
- Index creation uses `CREATE INDEX CONCURRENTLY` to avoid table locks on production. Alembic's `op.create_index` supports `postgresql_concurrently=True`.
- Schema changes and data migrations are separate files. Never combine them — schema changes are transactional; large data migrations must run outside a transaction (`op.execute("SET LOCAL lock_timeout = '5s'")`).

11.3 Running Migrations

In production, migrations run as a Kubernetes init container (or Docker Compose `depends_on` with a health check) before the application services start. The command: `alembic upgrade head`.

The application never auto-migrates on startup. Migrations are an explicit, reviewed, operator-triggered step.

---

12. Backup Strategy

12.1 PostgreSQL

Continuous WAL archiving: configure `archive_mode = on` and stream WAL to object storage (S3 or equivalent). This enables point-in-time recovery (PITR) to any second within the retention window.

Daily base backups: use `pg_basebackup` or a managed database service's snapshot feature. Retain daily backups for 30 days, weekly backups for 6 months.

Backup validation: weekly restore drill to a staging environment. Run `pg_dump | wc -l` and compare row counts on critical tables against the production database. Alert if counts diverge by more than 1%.

Retention by table:

- `users`, `auth_sessions`, `conversations`, `messages`, `scene_descriptions`: retain indefinitely (legal and user data requirements).
- `usage_events`: archive partitions older than 12 months to Parquet files in cold storage; drop the partition from PostgreSQL.
- `step_states`: retain while the corresponding `scene_descriptions` row exists.

12.2 Redis

Redis is a cache and operational store, not a system of record. The authoritative copy of all important data is in PostgreSQL. Redis data loss is recoverable:

- Conversation context (`llm_ctx:*`): if lost, the session resets. The conversation history in PostgreSQL is used to re-seed context if the user continues the session.
- Geometry cache (`geo:*`): if lost, the next request recomputes via Rust. Latency increases for the first request; subsequent requests re-populate the cache.
- Refresh tokens (`refresh:*`): if lost, all users are logged out. This is the only Redis data worth special treatment.

For refresh tokens (`DB 1`): enable Redis persistence with `AOF` (append-only file) mode set to `appendfsync everysec`. This limits data loss to at most 1 second of writes on failure. Back up the AOF file daily to object storage.

For operational keys (`DB 0`): no persistence. Accept data loss on Redis restart — the services handle cold-cache states gracefully.

---

13. Connection Pooling

13.1 PostgreSQL

Each Python service (LLM Service, Visualization Service, Auth Sidecar) uses SQLAlchemy 2.0 async with `asyncpg` as the driver.

Pool configuration per service instance:

- `pool_size`: 5 (minimum persistent connections)
- `max_overflow`: 10 (additional connections allowed under load)
- `pool_timeout`: 30 s (wait time for a connection from the pool before raising)
- `pool_recycle`: 1800 s (close and replace connections older than 30 min to avoid stale connections)

In production with multiple service instances, use **PgBouncer** in transaction-mode pooling in front of PostgreSQL. This multiplexes many application connections onto fewer PostgreSQL connections, which are expensive (each is an OS process). Target: PostgreSQL `max_connections = 100`; PgBouncer pool size = 20 per service type.

13.2 Redis

Each Python service uses `redis-py` async with a connection pool.

Pool configuration per service instance:

- `max_connections`: 20 for the Visualization Service (high geometry cache traffic), 10 for others.
- `socket_timeout`: 1 s (fast fail — Redis should always respond in < 100 ms).
- `socket_connect_timeout`: 2 s.
- `health_check_interval`: 30 s (keep-alive ping).

Use a single `redis.asyncio.ConnectionPool` shared across all requests within a service instance. Never create a new connection per request.

---

14. Security

14.1 Database Security

- The application database user has only `SELECT, INSERT, UPDATE, DELETE` on application tables. No `DROP`, `ALTER`, `CREATE`, or `TRUNCATE` permissions.
- A separate migration user has `DDL` permissions. It is only active during migration runs.
- `pg_hba.conf` restricts connections to the application subnet. No direct external access to PostgreSQL.
- All passwords stored with bcrypt, cost factor 12. Never SHA-256 or MD5.
- `hashed_pw` column is never returned by any query in application code — explicitly excluded from all `SELECT` statements using SQLAlchemy's `deferred()` loading.

14.2 Redis Security

- Redis requires authentication (`requirepass` set to a strong random password stored in the secrets manager).
- Redis is bound to the internal service network only (`bind 127.0.0.1 10.0.0.0/8`). Never exposed on a public interface.
- TLS enabled between application services and Redis using Redis's built-in TLS support (`tls-port`, `tls-cert-file`, etc.).
- The `KEYS *` and `FLUSHALL` commands are renamed to random strings in the Redis config (`rename-command KEYS ""`, `rename-command FLUSHALL ""`). This prevents accidental or malicious use.

14.3 Secrets Management

All secrets (JWT signing key, database passwords, Redis password, LLM API keys, internal service HMAC secret) are stored in a secrets manager (AWS Secrets Manager, HashiCorp Vault, or equivalent). Services fetch

secrets at startup and cache them in memory. Secrets are never written to environment files, Docker Compose `environment` blocks in source control, or application logs.

Secret rotation: the JWT signing key and internal HMAC secret rotate every 30 days via an automated rotation job. The grace window (JWT: 5 min, HMAC: 0 — immediate) is configured per secret type.

---

15. Monitoring and Observability

15.1 Health Checks

API Gateway (`/health`): returns 200 with `{"status": "ok"}`. Checked by the load balancer every 10 s.

Auth Sidecar (`/health`): validates Redis connectivity (PING) and PostgreSQL connectivity (SELECT 1). Returns 200 if both pass; 503 if either fails.

LLM Service and Visualization Service: each expose `/health` with the same Redis and PostgreSQL checks, plus a check that `mathviz_core` (Rust extension) is importable.

15.2 Key Metrics to Instrument

API Gateway: request rate, error rate (4xx/5xx), p50/p95/p99 latency, WebSocket connection count, rate limit hit rate.

Auth Sidecar: token validation latency, refresh token issuance rate, failed authentication rate (alert on spike — brute force indicator).

LLM Service: LLM provider call latency (p50/p95), Scene Description validation failure rate, self-correction retry rate, token usage per turn (cost tracking), provider error rate.

Visualization Service: Step 1 geometry latency (p95 target < 150 ms), Rust batch computation time (p95 target < 200 ms), Redis cache hit rate for geometry (`geo:*` keys), WebSocket connection duration distribution.

PostgreSQL: query latency by query pattern, connection pool utilization, replication lag (if using replicas), table bloat (from `pg_stat_user_tables`).

Redis: memory usage (alert at 80% of `maxmemory`), eviction rate, keyspace hit/miss ratio for `geo:*` namespace.

All metrics are exposed via Prometheus `/metrics` endpoints on each service and scraped by a Prometheus instance. Grafana dashboards are provided for each service layer.